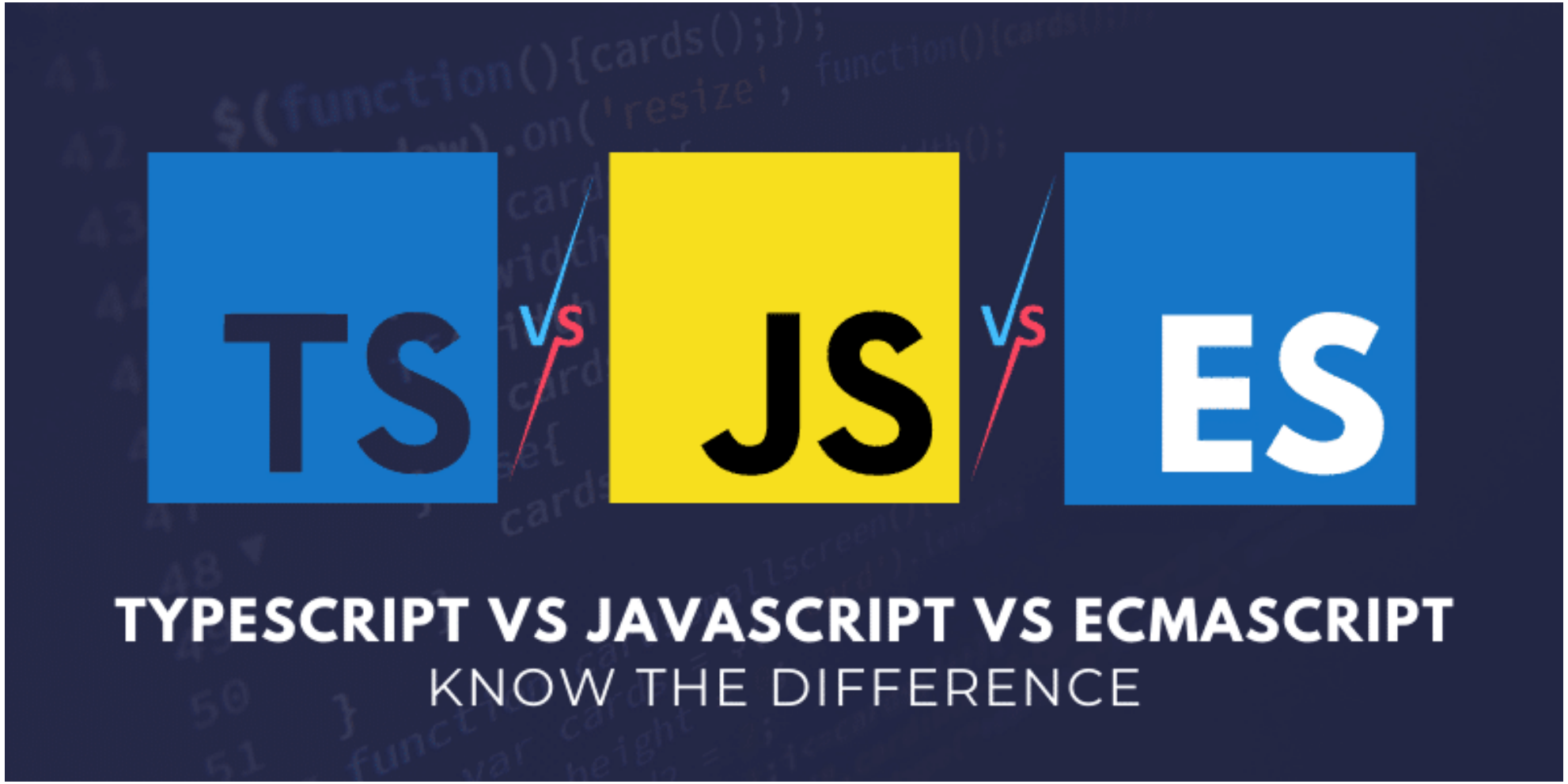
Programação e Algoritmos

Introdução ao Javascript e ao ambiente do Nodejs

Wagner Cesar Vieira
wagner.vieira01@cps.sp.gov.br

Agenda

- ✓ O que é JavaScript
- ✓ Formas de usar JavaScript
- ✓ Sintaxe da linguagem
- ✓ Relacionando elementos HTML no código JavaScript
- ✓ Funções nativas
- ✓ Exemplos práticos


TS
VS
JS
VS
ES
TYPESCRIPT VS JAVASCRIPT VS ECMASCRIPT
KNOW THE DIFFERENCE

O que é o Javascript?

JavaScript é uma linguagem de programação interpretada estruturada, de script em alto nível com tipagem dinâmica fraca e multiparadigma. Juntamente com **HTML** e **CSS**, o **JavaScript** é uma das três principais tecnologias da World Wide Web.

Recursos do JavaScript

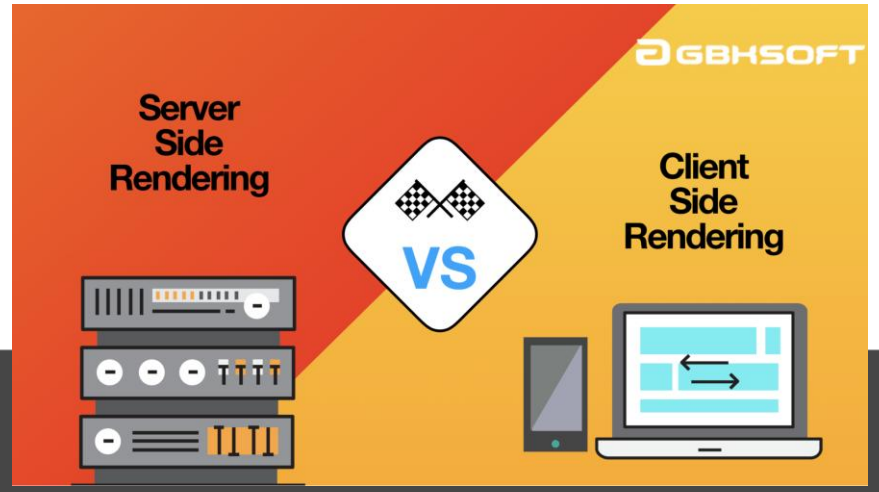
- É uma linguagem de programação com tipagem dinâmica;
- Pode ser colocada diretamente no arquivo HTML;
- Os navegadores interpretam a linguagem por conta própria;
- É uma das linguagens mais fáceis de aprender e mais versátil (front-end, back-end, mobile etc).

O que é o Javascript?

Você pode também ouvir os termos lado do **servidor** (**server-side**) e lado do **usuário** (**client-side**), especialmente no contexto de desenvolvimento web.

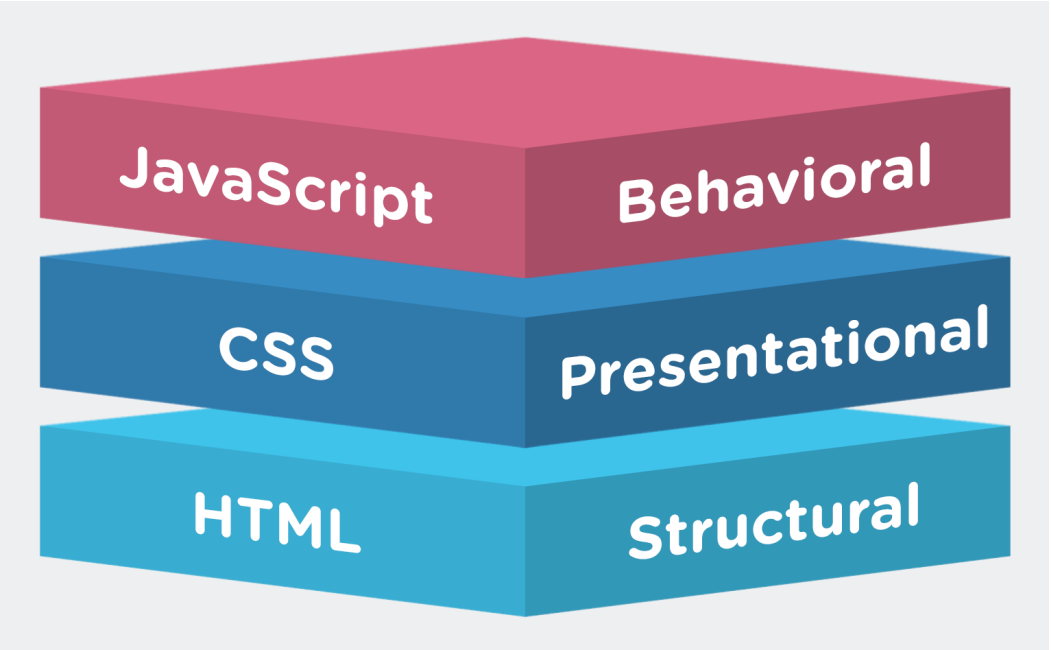
Códigos do lado do **client-side** são executados no computador do usuário — quando uma página web é visualizada, o código é baixado, executado e exibido pelo navegador.

Códigos do lado do **server-side**, por outro lado, são executados no servidor e o resultado da execução é baixado e exibido no navegador.



O que é o Javascript?

JavaScript é baseada em objetos. Trata suas estruturas básicas, propriedades do browser e os elementos de uma página HTML como objetos (entidades com propriedades e comportamentos) e permite que sejam manipulados através de eventos do usuário programáveis, operadores e expressões.



Variáveis

O identificador (**nome**) de uma variável segue a mesma regra de outras linguagens, isto é, deve começar por uma letra ou _

Os caracteres seguintes podem ser letras, números ou _ e não pode haver espaços ou caracteres especiais.

Declaração de variáveis

Antes do advento do ES6 (**EcmaScript 6**), as declarações de **var** eram a regra. Havia problemas associados à declaração de variáveis com var, no entanto. Por isso, foi necessário o surgimento de novas formas de declarar variáveis.

```
// declaração de variável usando var
var nome = 'Wagner' // o ; é opcional
nome = `Cesar` // atribui novo valor para a variável
console.log(nome) // imprime o valor da variável no console

var nome = "Vieira" // redeclara a variável com o mesmo nome
console.log(nome) //imprime o valor da variável no console
```

Escopo do var

Escopo, essencialmente, significa onde essas variáveis poderão ser utilizadas.

Declarações com var tem escopo global ou escopo de função/local.

```
// declaração de variável usando var
var mensagem = "Olá mundo!";

function novaFuncao() {
    var mensagem = 'Seja bem-vindo!!!';
}

console.log(mensagem); // Qual será a saída?
```

Escopo do let

let é, agora, a forma preferida de declaração de variáveis. Não é uma surpresa, já que ele é uma melhoria às declarações com var.

let tem escopo de bloco

Um bloco é uma porção de código cercado por { }. Um bloco vive dentro dessas chaves. Tudo o que estiver cercado por chaves é um bloco.

```
// declaração de variável usando let
let mensagem = "Olá mundo!";

function novaFuncao() {
    let mensagem = 'Seja bem-vindo!!!';
}

console.log(mensagem); // Qual será a saída?
```

Escopo do const

Variáveis declaradas com **const** mantêm valores constantes. Declarações com **const** compartilham algumas semelhanças com as declarações com **let**.

Declarações com const têm escopo de bloco

Assim como as declarações de let, as declarações de const somente podem ser acessadas dentro do bloco onde foram declaradas.

```
// declaração de variável usando const
const mensagem = "Olá mundo!";

function novaFuncao() {
    const mensagem = 'Seja bem-vindo!!!';
}

console.log(mensagem); // Qual será a saída?
```

Operadores aritméticos

- A maioria dos operadores de JavaScript são os mesmos de outras linguagens que seguem o padrão ECMA-262 (Java, ActionScript, PHP, C/C++, etc.);
- A seguir, listagem com os principais operadores por grupo e exemplos.

```
// Operador   Descrição          Exemplo
/*    +       Soma          */   let soma = 3 + 5; console.log('Soma: '+soma)
/*    -       Subtração     */   let subs = 3 - 5; console.log('Subtração: '+subs)
/*    *       Multiplicação */   let mult = 3 * 5; console.log('Multplicação: '+mult)
/*    /       Divisão       */   let div = 3 / 5; console.log('Divisão: '+div)

/*    %       Resto           */ let resto = 3 % 5; console.log('Resto da divisão: '+resto)
/*    **      Exponenciação   */ let expo = 3 ** 5; console.log('Exponenciação: '+expo)
/*    ++      Incremento      */ let inc = ++expo; console.log('Incremento: '+inc)
/*    --      Decremento      */ let dec = --expo; console.log('Decremento: '+dec)
```

Operadores de atribuição


```
// Operador  Descrição
/*    =      Recebe um valor     */
/*    +=     Recebe e adiciona   */
/*    -=     Recebe e subtrai    */
/*    *=     Recebe e multiplica */
/*    /=     Recebe e divide     */
/*    %=     Recebe o resto divisão */

//Exemplo
let val      = 10; console.log('Atribuição: ',val)
let val_mais = val += 2; console.log('Adicionado: ',val_mais)
let val_menos = val -= 2; console.log('Subtraído: ',val_menos)
let val_mult = val *= 2; console.log('Multiplicado: ',val_mult)
let val_div  = val /= 2; console.log('Dividido: ',val_div)
let val_rest = val %= 2; console.log('Restante: ',val_rest)
```

Operadores relacionais e lógicos

Operador	Nome	Ex.	Resultado
==	Igual	`a == b`	Verdadeiro se `a` for igual a `b`
!=	Diferente	`a != b`	Verdadeiro se `a` não for igual a `b`
===	Idêntico	`a === b`	Verdadeiro se `a` for idêntico a `b`
!==	Não Idêntico	`a !== b`	Verdadeiro se `a` não for igual a `b` ou eles não tiverem o mesmo tipo
<	Menor que	`a < b`	Verdadeiro se `a` for menor que `b`
>	Maior que	`a > b`	Verdadeiro se `a` for maior que `b`
<=	Menor ou igual	`a <= b`	Verdadeiro se `a` for menor ou igual a `b`
>=	Maior ou igual	`a >= b`	Verdadeiro se `a` for maior ou igual a `b`
&&	E	`a && b`	Verdadeiro se existir `a` e `b`
||	Ou	`a || b`	Verdadeiro se existir `a` ou existir `b`
!	Negação	`!a`	Negação, se o valor for verdadeiro vai transformar em falso e vice versa

```
Let a: number = 6;
Let b: number = 2;

a + b; //8
a - b; //4
a * b; //12
a / b; //3

a > b //true
a < b //false
a > b && b < a //true
a > b || b > a //true
```

Operadores especiais

Operador	Significado	Exemplo
?:	Segundo uma condição, retorna um valor ou outro	a ? b : c
in	Verifica se a propriedade especificada está no objeto especificado	title in pIntroducao
instanceof	Verifica se o objeto especificado é daquele tipo	x instanceof String
typeof	Retorna uma string indicando o tipo do operador	typeof "teste"
new	Permite criar um objeto a partir de um construtor	new String("teste")

Tipos de dados

- JavaScript aceita os tipos: number (inteiros ou com decimais), string, boolean e null;
- Entretanto, apresenta tipagem fraca, não sendo necessário declarar o tipo de uma variável e a mesma poderá receber valores de um tipo diferente, posteriormente.
- Nesse exemplo, a variável x é declarada inicialmente com um valor numérico e, sucessivamente, atribui-se a ela uma string, um boolean e, por fim, null.

Exemplo

```
let x = 0;
x = "Olá!";
x = true;
x = null;
```

Ambiente de desenvolvimento

Instalação do NodeJS – ambiente

https://nodejs.org/dist/v24.14.1/node-v24.14.1-x64.msi

Visual Studio

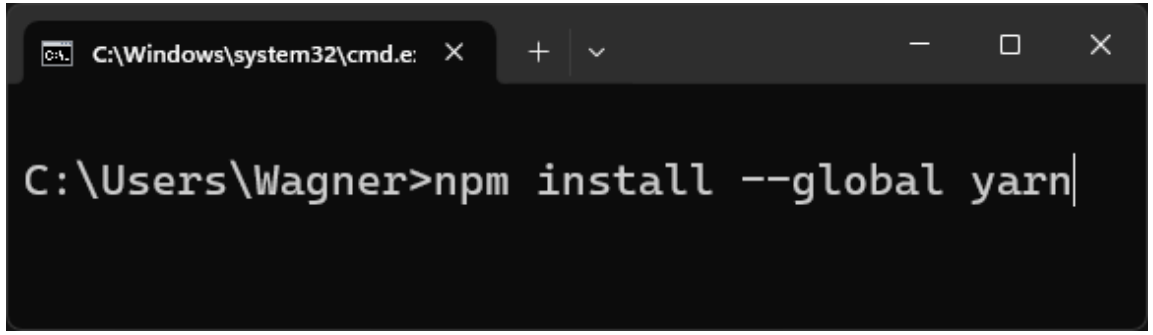
vs.

VS Code

NPM – Node Package Manager

O NPM (node package manager) é o gerenciador de pacotes do node.js (runtime de javascript que permite utilizar a linguagem no back-end).

Em primeiro lugar, com o NPM você pode gerenciar dependências do seu projeto, acessar o repositório do NPM e ter acesso a inúmeras bibliotecas e frameworks JavaScript, permite a instalação/desinstalação dos mesmos assim como a criação de seus próprios módulos públicos ou privados.




```
C:\Users\Wagner>node -v
v21.5.0

C:\Users\Wagner>npm -v
10.2.4

C:\Users\Wagner>yarn -v
1.22.21
```

NPM init – iniciando um projeto


```
PS D:\JQuery\setup_new> npm init
This utility will walk you through creating a package.json file.
It only covers the most common items, and tries to guess sensible defaults.

See `npm help init` for definitive documentation on these fields
and exactly what they do.

Use `npm install <pkg>` afterwards to install a package and
save it as a dependency in the package.json file.

Press ^C at any time to quit.
package name: (setup_new)
version: (1.0.0)
description: Ambiente de desenvolvimento
entry point: (index.js)
test command:
git repository:
keywords:
author: Wagner Cesar Vieira
license: (ISC)
About to write to D:\JQuery\setup_new\package.json:
```

Arquivo package.json criado


```
{
  "name": "setup_new",
  "version": "1.0.0",
  "description": "Ambiente de desenvolvimento",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "Wagner Cesar Vieira",
  "license": "ISC"
}


Is this OK? (yes) yes
```

```
PS D:\JQuery\setup_new> dir


    Diretório: D:\JQuery\setup_new


Mode                LastWriteTime         Length Name
----                -------------         ------ ----
-a----        02/02/2023     16:00            251 package.json
```

Gerenciando módulos com NPM

- **npm install** nome_do_módulo: instala um módulo no projeto.
- **npm install -g** nome_do_módulo: instala um módulo global.
- **npm install** nome_do_módulo **--save**: instala o módulo no projeto, atualizando o package.json na lista de dependências.
- **npm list**: lista todos os módulos do projeto.
- **npm list -g**: lista todos os módulos globais.
- **npm remove** nome_do_módulo: desinstala um módulo do projeto.

Gerenciando módulos com NPM

- **npm remove -g** nome_do_módulo: desinstala um módulo global.
- **npm update** nome_do_módulo: atualiza a versão do módulo.
- **npm update -g** nome_do_módulo: atualiza a versão do módulo global.
- **npm -v**: exibe a versão atual do npm.

Entendendo o package.json

Todo projeto Node.js é chamado de módulo. O termo módulo surgiu do conceito de que a arquitetura do Node.js é modular. E todo módulo é acompanhado de um arquivo descritor, conhecido pelo nome de **package.json**.

Este arquivo é essencial para um projeto Node.js. Um **package.json** mal escrito pode causar bugs ou impedir o funcionamento correto do seu módulo, pois ele possui alguns atributos chaves que são compreendidos pelo Node.js e NPM.

package.json > ...

```
1  {
2    "name": "variaveis",
3    "version": "1.0.0",
4    "description": "",
5    "main": "index.js",
     ▷Debug
6    "scripts": {
7      "test": "echo \"Error: no test specified\" && exit 1"
8    },
9    "keywords": [],
10   "author": "",
11   "license": "ISC",
12   "type": "module"
13 }
```

Wagner Cesar Vieira
wagner.vieira01@cps.sp.gov.br